

Handbook Module 1

Artificial Intelligence

Module 1: Intro to AI and Python Ecosystem

Written By: Zahran Alvan Putra Winarko, GDGOC ITB Curriculum Team

The Modern AI Landscape

What is Artificial Intelligence

Artificial intelligence (AI) is the ability of a computer system to mimic human cognitive functions such as recognising patterns, understanding language, making decisions, and learning from experience. The fundamental difference from conventional programs: a regular program follows instructions written explicitly by humans, while AI infers its own instructions from data.

A brief history, the term artificial intelligence was first coined by John McCarthy at the 1956 Dartmouth Conference. The following decades alternated between waves of optimism and AI winters. A major turning point came in 2012 when AlexNet demonstrated that deep learning could beat classical techniques in image classification by an enormous margin. Since then, AI development has followed an exponential curve culminating in large language models (LLMs) such as GPT, Gemini, and Claude.

Reference: [What is Artificial Intelligence \(AI\)?](#)

AI, Machine Learning, Deep Learning, and Generative AI

These four terms are often used interchangeably incorrectly. Their relationship is a nested subset hierarchy, from the broadest to the most specific.

- AI: The broadest eld. Encompasses all systems that mimic human intelligence including rule-based systems, expert systems, and ML.
- Machine Learning: An AI subset where a system learns from data without being explicitly programmed for every case. The model optimises its parameters based on patterns in training data.
- Deep Learning: An ML subset that uses multi-layer neural networks (deep neural networks). Capable of handling raw unstructured data (images, text, audio) directly.
- Generative AI: A DL subset focused on creating new content text, images, code, audio that did not previously exist. Examples: LLMs (GPT, Gemini), image generators (Stable Diffusion).

It is also important to distinguish two approaches within ML:

- Classical ML (non-deep): features must be manually engineered by a domain expert before being fed into the model. Examples: Logistic Regression, Random Forest, SVM.
- Deep Learning: the model automatically learns feature representations from raw data. Requires more data but is more flexible for complex problems.

Reference:

[AI vs. machine learning vs. deep learning vs. neural networks: What's the difference?](#)

[Machine Learning Crash Course](#)

AI Taxonomy: Narrow, General, Super

Based on capability scope, AI is divided into three levels:

- Artificial Narrow Intelligence (ANI): designed for one specific task. This is what exists and is used every day AlphaGo for chess/Go, Netflix recommendation systems, GPT-4 for text. Despite appearing intelligent, ANI cannot transfer its knowledge to other domains.
- Artificial General Intelligence (AGI): a hypothetical AI capable of performing any task a human can, including abstract reasoning and cross-domain transfer learning. Not yet achieved. There is vigorous debate in the research community about when (or whether) AGI will be reached.
- Artificial Super Intelligence (ASI): a hypothetical AI that surpasses human intelligence in every dimension. This concept belongs to the realm of AI safety research, not current practical engineering.

Reference: [What is Artificial Intelligence \(AI\)?](#)

AI Applications and Ethics

AI in the Real World (2026)

AI is embedded in almost every layer of technology we use today:

- Content Recommendation: YouTube, Spotify, TikTok, and Netflix use collaborative filtering and deep learning models to personalise every user's feed.

- Conversational Assistants: Gemini, Claude, ChatGPT used for coding, research, brainstorming, and content writing.
- Computer Vision: Face ID on iPhone, licence plate recognition in parking systems, object detection for self-driving cars (Waymo, Tesla Autopilot).
- Healthcare: AI for radiology analysis (tumour detection from X-ray and MRI), sepsis prediction in ICUs, drug discovery.
- Finance: real-time fraud detection, credit scoring, algorithmic trading.
- AI-Assisted Coding: Gemini Code Assist, Antigravity, Claude Code, GitHub Copilot, Cursor models that help engineers write, debug, and refactor code.

Reference: [What is Artificial Intelligence \(AI\)?](#)

Hallucination, Bias, and Real-World Risks

AI is not without risk. Understanding those risks is an essential part of being a responsible AI practitioner.

Hallucination

LLMs sometimes produce information that sounds convincing but is factually wrong this is called hallucination. The cause: models are trained to generate the statistically most probable next token, not the most truthful one. A model does not know what it does not know, so it continues generating text even without valid grounding. Real-world consequences: fabricated legal precedents, non-existent journal citations, incorrect medical diagnoses. This is why LLM outputs must always be verified for high stakes use cases.

Algorithmic Bias

Models learn from human-generated data. If the historical data contains bias (e.g., recruitment data that historically favoured men for technical roles, or healthcare data that underrepresents certain ethnic groups), the model will replicate and even amplify that bias. A well-documented real example: an AI-based recruitment system that systematically penalised female candidates because it was trained on historically gender-biased hiring patterns.

Prompt Injection and Security

Prompt injection is an attack where malicious instructions are embedded inside input that an LLM processes, causing the model to perform actions outside the

developer's intent. Example: hidden text inside a document that instructs the AI to ignore its system prompt and exfiltrate sensitive data. This is not a theoretical problem, it has occurred in production systems.

Relevance for Vibe Coding: When using AI coding tools, never paste credentials, API keys, or sensitive user data directly into a prompt. Models may store or log that data depending on the provider's policy.

Responsible AI – Google's Framework

Google defines three core principles for responsible AI development:

- **Bold Innovation:** develop AI that delivers substantial benefits while managing foreseeable risks. This is not “don't build harmful AI” it is “ensure that the overall benefit is substantially greater than the risk”.
- **Responsible Development and Deployment:** apply human oversight across the entire lifecycle from design, training, and evaluation through deployment and postrelease monitoring. This includes red-teaming and bias evaluation before any public release.
- **Collaborative Approach:** AI safety research is conducted jointly with the academic community, regulators, and society at large. It cannot be done unilaterally by a single company.

Beyond Google's framework, several principles are widely adopted across the industry:

- fairness (outputs must be equitable across all groups),
- accountability (someone must be responsible for AI decisions),
- transparency (users should know when they are interacting with AI), and
- safety (models must not be used to cause harm).

Reference:

[Responsible AI Practice from Google AI](#)

[Ethical and social risks of harm from Language Models](#)

Python for AI Pipelines

Why Python?

Python dominates the AI ecosystem not because it is the fastest (C++ and Rust are faster), but because of three factors:

- Library ecosystem: NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch, Hugging Face Transformers, all have first-class Python support.
- Rapid iteration: expressive syntax and REPL environments (Jupyter Notebook, Colab) enable interactive data exploration without a compile cycle.
- Interoperability: Python wraps C/C++ code easily (NumPy itself is implemented in C), so we get convenient syntax without sacrificing performance at the computational layer.

Data Structures Relevant to ML

Not all Python data structures appear with the same frequency in ML pipelines. Focus on the ones that actually matter:

1. List – The General-Purpose Collector

```
# Common use: storing a batch of items before converting to
an array
labels = ['cat ', 'dog ', 'cat ', 'bird ']
scores = [0.92 , 0.87 , 0.95 , 0.78]

# List comprehension: a fast, idiomatic transformation
pattern
normalized = [ s / max( scores ) for s in scores ]

# Mutable and ordered , but SLOW for large numerical
operations

# -> convert to a NumPy array when you need intensive
computation
```

2. Dictionary – Metadata and Configuration

```
# Very common for storing model hyperparameters
config = {
    'learning_rate ': 0.001,
    'batch_size ': 32,
    'epochs ': 50,
    'dropout ': 0.3,
}
```

```
# Also used for label encoding
label_map = {'cat ': 0 , 'dog ': 1 , 'bird ': 2}
encoded = [ label_map [ l ] for l in labels ]
```

3. Tuple – Immutable Data and Shapes

```
import numpy as np

arr = np . zeros ((3 , 224 , 224) ) # ( channels , height ,
width ), shape is a tuple
print ( arr . shape ) # (3 , 224 , 224), numpy . shape
returns a tuple

# Tuples can be dictionary keys ( hashable , unlike lists )
cache = {}
cache [(224 , 224) ] = arr
```

4. Set – Deduplication and Fast Lookup

```
# Useful for building vocabulary sets in NLP
vocabulary = set ([ 'the ', 'cat ', 'sat ', 'on ', 'the ',
'mat '])

print ( vocabulary ) # {'the ', 'cat ', 'sat ', 'on ', 'mat
'} # duplicates removed

# O(1) membership check, far faster than a list for large
vocabularies
stop_words = {'the ', 'a', 'an ', 'is ', 'are '}
filtered = [ w for w in ['the ', 'cat ', 'is ', 'fast '] if w
not in stop_words ]
```

Functional Programming Patterns in Data Processing

1. Lambda, map(), and filter()

```
# Lambda : anonymous single - expression functions
double = lambda x : x * 2

# map (): apply a function to every element
```

```
values = [1 , 2 , 3 , 4 , 5]
doubled = list (map( lambda x : x * 2 , values ) ) # [2 , 4 ,
6 , 8 , 10]

# filter (): keep only elements that satisfy a condition
positives = list ( filter ( lambda x : x > 0 , [ -1 , 2 , -3
, 4 , 0] ) ) # [2 , 4]

# In the context of Pandas , lambda is frequently used with .
apply ()
import pandas as pd
df = pd . DataFrame ({ 'score ': [0.9 , 0.4 , 0.7 , 0.3]})
df ['grade '] = df ['score ']. apply ( lambda x : 'PASS ' if
x >= 0.5 else 'FAIL ' )
```

2. Modular Functions for Pipelines

```
# A common ML pattern : break the pipeline into small ,
independently testable
# functions that can be composed together .

def load_data ( path ) :
    return pd . read_csv ( path )

def clean_data ( df ) :
    df = df . dropna ()
    df = df [ df ['value '] > 0]
    return df

def normalize ( df , columns ) :
    for col in columns :
        df [ col ] = ( df [ col ] - df [ col ]. mean () ) / df
        [ col ]. std ()
    return df

# The pipeline reads almost like a sentence :
# df = normalize ( clean_data ( load_data ( ' data .csv ' ) ) ,
[' feature_1 ', ' feature_2 '])
```

NumPy Efficient Numerical Computation

NumPy is the foundation of numerical computation in Python. NumPy arrays (ndarray) differ from Python lists in a critical way: operations on NumPy arrays are executed in compiled C code rather than the Python interpreter, making them 10x100x faster for large-scale numerical operations.

1. Creating Arrays

```
import numpy as np

# From a list
arr = np . array ([1 , 2 , 3 , 4 , 5])
matrix = np . array ([[1 , 2 , 3] , [4 , 5 , 6]])

# Utility constructors
np . zeros ((3 , 4) ) # 3x4 array of zeros
np . ones ((2 , 3) ) # 2x3 array of ones
np . eye (4) # 4x4 identity matrix
np . arange (0 , 10 , 2) # [0 2 4 6 8]
np . linspace (0 , 1 , 5) # [0. 0.25 0.5 0.75 1. ]
np . random . randn (100 , 10) # 100 x10 from a standard normal distribution
```

2. Indexing, Slicing, and Boolean Masking

```
arr = np . arange (10) # [0 1 2 3 4 5 6 7 8 9]
# Slicing : array [ start : stop : step ]
print ( arr [2:7]) # [2 3 4 5 6]
print ( arr [::2]) # [0 2 4 6 8]
print ( arr [::-1]) # [9 8 7 6 5 4 3 2 1 0]

# 2D arrays
M = np . array ([[1 ,2 ,3] , [4 ,5 ,6] , [7 ,8 ,9]])
print ( M [1 , :]) # [4 5 6] # second row
print ( M [: , 0]) # [1 4 7] # first column
print ( M [:2 , :2]) # [[1 2] [4 5]] # top - left 2x2 submatrix

# Boolean masking, a pattern used constantly in real ML code
scores = np . array ([0.9 , 0.4 , 0.7 , 0.3 , 0.85])
```

```
high = scores [ scores > 0.6] # [0.9 0.7 0.85]
```

View vs Copy

NumPy slicing returns a view of the original array, not a copy. Modifying the slice modifies the original. Use `.copy()` when you need an independent copy.

3. Vectorised Operations and Broadcasting

```
a = np . array ([1 , 2 , 3 , 4])
b = np . array ([10 , 20 , 30 , 40])

# Vectorised : the operation is applied to all elements at
once
print ( a + b ) # [11 22 33 44]
print ( a * b ) # [10 40 90 160]
print ( np . dot ( a , b ) ) # dot product : 300

# Broadcasting : operations between arrays of different
shapes
M = np . array ([[1 ,2 ,3] ,[4 ,5 ,6]])
row = np . array ([10 , 20 , 30])
print ( M + row ) # [[11 22 33] [14 25 36]] # row broadcast
across M

# Common statistical functions
data = np . random . randn (1000)
print ( data . mean () , data . std () , data .min () , data
.max () )
print ( np . percentile ( data , [25 , 50 , 75]) )

# Axis - wise reductions ( axis =0 -> per column , axis =1 ->
per row)
M = np . array ([[1 ,2 ,3] ,[4 ,5 ,6] ,[7 ,8 ,9]])
print ( M .sum( axis =0) ) # [12 15 18] # column sums
print ( M . mean ( axis =1) ) # [2. 5. 8.] # row means
```

Reference:

[NumPy - Learn](#)

[Python Numpy Tutorial \(with Jupyter and Colab\)](#)

Pandas introduces two core data structures built on top of NumPy: Series (a labelled 1D array) and DataFrame (a labelled 2D table). Where NumPy excels at pure numerical computation, Pandas excels at mixed tabular data (numbers, strings, datetimes).

1. DataFrame Basics

```
import pandas as pd

# Create from a dictionary
df = pd . DataFrame ({
    'name ': ['Alice ', 'Bob ', 'Charlie '],
    'score ': [0.92 , 0.78 , 0.85] ,
    'grade ': ['A', 'B', 'A'] ,
})

# Initial inspection, always do this after loading a new dataset
print ( df . shape ) # (3 , 3)
print ( df . dtypes ) # data type of each column
print ( df . info () ) # summary including non - null counts
print ( df . describe () ) # descriptive statistics for numeric columns
print ( df . head (10) ) # first 10 rows
```

2. Loading Data from Files

```
# CSV, the most common format
df = pd . read_csv ('data .csv ')
df = pd . read_csv ('https :// example . com / data .csv ') # works with URLs too

# Useful arguments
df = pd . read_csv ('data .csv ',
    sep=';', # if the delimiter is not a comma
    encoding='utf -8 ',
    usecols =[ 'col1 ', 'col2 '], # load only specific columns
    nrows =1000 , # load only the first N rows 11 )
```

```
# Save to CSV
df . to_csv ('output .csv ', index = False ) # index = False
avoids writing the row index
```

3. Selection and Filtering

```
# Column selection
col = df ['score '] # single column as Series
subset = df [[ 'name ', 'score ']] # multiple columns as
DataFrame
```

```
# Row filtering
high = df [ df ['score '] > 0.8]
passing_a = df [( df ['score '] > 0.8) & ( df ['grade '] ==
'A') ]
```

```
# loc: label - based indexing
df . loc [0 , 'name '] # row 0 , column 'name '
df . loc [ df ['score '] > 0.8 , 'name '] # filter rows +
select column
```

```
# iloc : integer - position - based indexing
df . iloc [0 , :] # first row , all columns
df . iloc [:5 , :2] # first 5 rows , first 2 columns
```

4. Groupby and Aggregation

```
# Groupby implements a split -apply - combine strategy
# Example : mean score per grade
result = df . groupby ('grade ') ['score ']. mean ()
```

```
# Multiple aggregation functions at once
stats = df . groupby ('grade ') . agg (
    avg_score = ('score ', 'mean ') ,
    count = ('name ', 'count ') ,
    max_score = ('score ', 'max ') ,
)
```

```
# Groupby on multiple columns
# df. groupby ([' col1 ', 'col2 '])[' target ']. mean ()
```

5. Handling Missing Values

```
# Identify missing data
print ( df . isnull () .sum () ) # number of NaN values per
column
print ( df . isnull () . mean () ) # proportion of NaN per
column

# Handling strategies
df . dropna () # drop all rows containing NaN
df . dropna ( subset =[ 'score ']) # drop only if 'score ' is
NaN
df . fillna (0) # fill NaN with 0
df ['score ']. fillna ( df ['score ']. median () ) # fill
with median ( more robust )
```

Reference:

[Pandas User Guide](#)

[Using pandas and Python to Explore Your Dataset](#)

[Learn Pandas | Kaggle](#)

AI-Assisted Development: Vibe Coding

What Is Vibe Coding?

Vibe coding is a software development approach where the programmer describes what they want to accomplish in natural language or pseudocode, and an AI coding tool translates that description into concrete code. The term was popularised by Andrej Karpathy in early 2025. Crucially, vibe coding does not mean you no longer need to understand the code. You still need to read, evaluate, and modify the AI's output. AI is a co-pilot, not an autopilot. The most effective engineers in this era are those who can direct AI with precision while also detecting errors that AI introduces.

Tool Overview: Gemini Code Assist, Antigravity, Claude Code, GitHub Copilot, Cursor

- Gemini Code Assist: Integrated with the Google ecosystem, free for individual use, works well inside Colab.

- **Antigravity:** Google's autonomous agent-based development platform features a standalone IDE. Its AI can run complex tasks in parallel across workspace projects. It can also read filesystems.
- **Claude Code:** Anthropic's autonomous CLI (Command Line Interface) tool that operates directly within the terminal, this AI agent can read directory structures, edit files, run testing commands, and perform Git commits automatically.
- **GitHub Copilot:** Seamlessly integrated in VS Code, wide code context window, fast tab completion.
- **Cursor:** A full IDE built on VS Code with AI deeply integrated; can chat with the entire codebase.

Working Effectively with AI Coding Tools

The core principle: the more specific your prompt, the more useful the output.

BAD : too vague

"write a pandas function "

GOOD : specific , includes context , states expected output 5

"Write a Python function that takes a DataFrame with columns 'genre ' and 6 'popularity ' , and returns a new DataFrame containing the mean popularity 7 per genre sorted from highest to lowest . Use Pandas groupby . Include 8 type hints ."

Effective usage patterns:

- **Code generation:** describe the function or class you need, let the AI draft it, then review and modify.
- **Debugging:** paste the error message alongside the relevant code and ask the AI to explain the root cause and suggest a fix.
- **Refactoring:** paste working code and ask the AI to make it cleaner, more efficient, or more readable.
- **Code explanation:** paste code you do not understand (e.g., from an unfamiliar library) and ask the AI to walk through it line by line.

Things to Avoid

- Pasting AI-generated code without reading it you are responsible for every line you submit.
- Trusting AI output for critical numerical computations without verifying.
- Pasting API keys, passwords, or users' personal data into prompts .

Reference:

[GitHub Copilot Documentation](#)

[Gemini Code Assist Overview](#)